A. Java Collection Framework (35 Questions)

1. What is the Java Collection Framework?

The Java Collection Framework (JCF) is a set of interfaces, classes, and utility methods for storing and working with groups of objects.

It provides:

- **Interfaces** that describe collection behavior, such as List, Set, Queue, and Map.
- **Implementations** that provide specific ways to store data, such as ArrayList, HashSet, PriorityQueue, and HashMap.
- **Algorithms and utilities** in the Collections class, such as sorting, searching, reversing, and shuffling lists.

For example, a List interface describes an ordered collection that can contain duplicates, while ArrayList is one implementation of that interface.

List<String> names = new ArrayList<>();

names.add("Asha");

names.add("Ravi");

Using an interface type such as List makes code easier to change: you can later use a different implementation, such as LinkedList, without changing every line that uses the variable.

2. Why do we use collections instead of arrays?

Arrays are useful when the number of elements is fixed and you want a simple, efficient structure. Collections are more convenient when you need flexible size or built-in operations.

Collections can:

- Grow or shrink as elements are added or removed.
- Provide methods for searching, sorting, iteration, and other common operations.
- Offer different storage and behavior choices, such as ordered lists, sets of unique elements, or queues.
- Use generics to make the types of stored objects clearer.

Example:

```
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.remove("Asha");
```

An array has a fixed length after creation:

```
String[] names = new String[2];
```

collections usually store objects, so primitive values use wrapper types such as Integer instead of int. Java's autoboxing often makes the conversion automatic:

List<Integer> numbers = new ArrayList<>();

numbers.add(10); // int is boxed as an Integer

3. What is the difference between Collection and Collections?

Collection and Collections are different things.

- **Collection** is an interface that represents a group of objects. Interfaces such as List, Set, and Queue extend it.
- **Collections** is a utility class with static methods for working with collections.

Aspect	Collection	Collections
What it is	An **interface** in the Java Collection Framework	A **utility class** in java.util
Purpose	Represents a group of objects	Provides static methods for working with collections
Examples	List, Set, and Queue extend Collection	Collections.sort(), Collections.reverse(), Collections.shuffle()
Can you create it directly?	No; create an implementation such as ArrayList	No; its constructor is private
Example	Collection<String> names = new ArrayList<>();	Collections.sort(names);

Collection<String> items = new ArrayList<>();
items.add("pear");

List<Integer> numbers = new ArrayList<>(List.of(3, 1, 2));
Collections.sort(numbers);

4.What is the difference between Collection and Map?
A Collection holds individual elements. A Map holds associations between keys and values.

Aspect	Collection	Map
Stores	Individual elements	Key-value pairs
Example	["apple", "banana"]	{"Asha" = 90, "Ravi" = 85}
Duplicate rules	Depend on the type: a List allows duplicates; a Set does not	Keys must be unique; values can repeat
Access	Often by iteration, or by index for a List	Look up a value using its key
Main interfaces/classes	List, Set, Queue	Map, HashMap, TreeMap, LinkedHashMap
Example code	Collection<String> fruits = new ArrayList<>();	Map<String, Integer> scores = new HashMap<>();

Aspect	Collection	Map
Framework relationship	Collection is an interface	Map is also an interface, but it does **not** extend Collection

5. What is the difference between List, Set, and Queue?

These interfaces describe different collection behaviors:

Feature	List	Set	Queue
Stores	An ordered sequence of elements	Unique elements	Elements waiting to be processed
Allows duplicates?	Yes	No	Usually yes, depending on implementation
Order	Preserves a defined sequence; supports positions by index	Depends on implementation: HashSet has no guaranteed order, LinkedHashSet preserves insertion order, and TreeSet sorts	Depends on implementation: often FIFO, but PriorityQueue uses priority order
Access	Can access elements by index with get(index)	Usually checks membership with contains(); no index	Usually examines or removes the head with peek() or poll()
Common implementations	ArrayList, LinkedList	HashSet, LinkedHashSet, TreeSet	ArrayDeque, LinkedList, PriorityQueue
Typical use	Keep an ordered collection, including repeated items	Prevent duplicates	Process items in an order, such as tasks or messages
Example	List<String> names = new ArrayList<>();	Set<String> names = new HashSet<>();	Queue<String> tasks = new ArrayDeque<>();

6. What is the difference between ArrayList and LinkedList?

ArrayList is backed by a resizable array. LinkedList is a doubly linked list.

Operation	ArrayList	LinkedList
Access by index	Usually fast, approximately O(1)	Requires traversal, O(n)
Add at the end	Usually fast; occasional resizing	Usually O(1)

Operation	ArrayList	LinkedList
Insert/remove near the beginning or middle	May shift many elements, O(n)	Relinking is quick after reaching the position, but reaching it can take O(n)
Memory overhead	Generally lower	Higher, because each node stores links

A LinkedList is not automatically faster for insertions and removals at arbitrary positions. Finding the position can still take time. In many everyday cases, ArrayList is a good default because it offers fast indexed access and lower overhead.

Both classes implement List. LinkedList also implements Deque, so it can be used as a double-ended queue.

7. What are the important features of ArrayList?

ArrayList is a resizable list implementation. Its main features include:

- It preserves the order of elements.
- It allows duplicates and null elements.
- It supports access by index.
- It grows as elements are added.
- It is not synchronized by default.
- Reading an element by index is usually fast; inserting or removing elements in the middle may require shifting later elements.

```
List<String> colors = new ArrayList<>();

colors.add("blue");

colors.add("green");

colors.add("blue"); // duplicates are allowed
```

If multiple threads modify an ArrayList at the same time, the program needs an appropriate thread-safety strategy. Merely sharing the list does not make it thread-safe.

8. How do you add elements to an ArrayList?

Use add(element) to append an element, or add(index, element) to insert it at a particular position.

```
List<String> cities = new ArrayList<>();

cities.add("Delhi");

cities.add("Pune");

cities.add(1, "Jaipur"); // insert at index 1

System.out.println(cities); // [Delhi, Jaipur, Pune]
```

The index for insertion must be between 0 and the current size, inclusive. Adding at the end is usually the most common operation.

9. What is the difference between add() and add(index, element)?

feature	add()	add(index, element)
What it does	Adds the element to the end of the list	Inserts the element at the specified index
Existing elements	Stay in their current positions	At that index and after it, elements shift right
Return value	Returns true	Returns void
Valid index	Not applicable	From 0 to size() inclusive
Example	list.add("C");	list.add(1, "B");

10. How do you access an element from an ArrayList?

Use get(index). List indexes start at 0.

List<String> fruits = new ArrayList<>(List.of("apple", "mango", "orange"));

String secondFruit = fruits.get(1); // mango

If the index is negative or is greater than or equal to the list size, get() throws IndexOutOfBoundsException.

11. How do you remove an element from an ArrayList?

You can remove by index or by object value.

List<String> fruits = new ArrayList<>(List.of("apple", "mango", "orange"));

fruits.remove(1); // removes the element at index 1: mango

fruits.remove("orange"); // removes a matching object

The index version returns the removed element. The object version returns true if a matching element was removed, otherwise false.

12. What is the difference between remove(int index) and remove(Object)?

Feature	remove(int index)	remove(Object)
What it removes	The element at the specified position	The first element equal to the specified object
Return value	Returns the removed element	Returns true if an element was removed; otherwise false
Example with strings	list.remove(1); removes the item at index 1	list.remove("apple"); removes "apple"
Index/value behavior	The argument is treated as an index	The argument is treated as an object value

Special case: With List<Integer>, an integer argument selects the index version:

List<Integer> numbers = new ArrayList<>(List.of(10, 20, 30));

```
numbers.remove(1);              // removes the item at index 1: 20
numbers.remove(Integer.valueOf(30)); // removes the value 30
```

13. How do you find the size of an ArrayList?

Use size(). It returns the number of elements currently in the list.

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi"));
int count = names.size(); // 2
```

size() is not the same as an array's length field. For an array, use array.length; for a collection, use collection.size().

14. How do you check whether an ArrayList contains a particular element?

Use contains(element). It returns true if the list contains an element equal to the given object, based on equals().

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi"));
if (names.contains("Ravi")) {
  System.out.println("Found Ravi");
}
```

For a custom class, the result depends on that class's equals() implementation. If the class uses the default Object.equals(), matching generally means the same object instance.

15. How do you iterate through an ArrayList?

Common approaches include an enhanced for loop, an index-based loop, an iterator, and forEach().

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi"));
// Enhanced for loop
for (String name : names) {
  System.out.println(name);
}
// Index-based loop
for (int i = 0; i < names.size(); i++) {
  System.out.println(names.get(i));
}
// forEach
names.forEach(System.out::println);
```

Use an index-based loop when you need the position. Use an Iterator when you need to remove elements safely during traversal:

```
Iterator<String> iterator = names.iterator();
while (iterator.hasNext()) {
```

```
    if (iterator.next().startsWith("A")) {
        iterator.remove();
    }
}
```

16. What is a Set?

A Set is a collection that does not allow duplicate elements according to its equality rules. It is useful when uniqueness matters, such as storing distinct usernames or tags.

```
Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("collections");
tags.add("java"); // duplicate; the set still contains just one "java"
```

17. What is the main difference between List and Set?

Feature	List	Set
Duplicates	Allows duplicate elements	Does not allow duplicate elements
Order	Maintains a defined sequence	Depends on implementation
Access	Can access elements by index	No index-based access
Common implementations	ArrayList, LinkedList	HashSet, LinkedHashSet, TreeSet
Example use	Keeping an ordered list of names, including repeats	Keeping unique names

18. Does a HashSet allow duplicate elements?

No. A HashSet does not retain duplicate elements according to equals() and hashCode().

```
Set<String> words = new HashSet<>();
words.add("hello");
words.add("hello");
System.out.println(words.size()); // 1
```

For custom objects, hashCode() and equals() must be implemented consistently for logically equal objects to be treated as duplicates. In particular, equal objects must return the same hash code.

19. Does a HashSet maintain insertion order?

No. HashSet makes no guarantee about iteration order. The order can differ from the insertion order and may change as the set changes or the runtime details vary.

If insertion order matters, use LinkedHashSet. If sorted order matters, use TreeSet.

20. What is the difference between HashSet and LinkedHashSet?

Feature	HashSet	LinkedHashSet
Duplicates	Does not allow duplicates	Does not allow duplicates
Iteration order	No guaranteed order	Preserves insertion order
Performance	Usually slightly faster	Usually slightly slower due to order tracking
Memory use	Generally lower	Generally higher because it tracks insertion order
Common use	When order does not matter	When elements should be unique and iterated in insertion order

21. What is the difference between HashSet and TreeSet?

Feature	HashSet	TreeSet
Duplicates	Does not allow duplicates	Does not allow duplicates
Iteration order	No guaranteed order	Sorted order
How order is determined	No ordering; uses hashing	Natural ordering or a supplied Comparator
Requirements for elements	Must implement hashCode() and equals() consistently	Must be comparable, or the set must have a Comparator
null elements	Allows one null element	Usually does not allow null with natural ordering
Range operations	No built-in sorted range operations	Supports operations such as first(), last(), and subSet()
Common use	Fast membership checks when order does not matter	Unique elements in sorted order or range queries
Example	Set<Integer> set = new HashSet<>();	Set<Integer> set = new TreeSet<>();

22. What is the purpose of TreeSet?

TreeSet is used when you need a set of unique elements that can be traversed in sorted order.

It can also provide range-related operations through NavigableSet, such as finding the nearest lower or higher element, or retrieving a subset in a range.

NavigableSet<Integer> scores = new TreeSet<>();

scores.add(72);

```
scores.add(95);
scores.add(84);
System.out.println(scores);        // [72, 84, 95]
System.out.println(scores.higher(84)); // 95
```

Elements must be naturally comparable, or the set must be constructed with a suitable Comparator.

23. How does TreeSet maintain its elements?

TreeSet keeps elements sorted using a self-balancing tree structure. The ordering comes from either:

1. The elements' natural ordering, usually through Comparable, or
2. A Comparator provided when the set is created.

Adding, removing, and searching typically take O(log n) time.

Because the set uses comparisons to establish order and uniqueness, a comparison that returns 0 means the elements occupy the same position in the set. This can cause one element to be treated as equivalent to another even if their equals() methods say otherwise. It is best for the ordering to be consistent with equals().

24. What is a Queue?

A Queue is a collection designed to hold elements before they are processed. Many queue implementations process elements in first-in, first-out (FIFO) order, but not all do. For example, a PriorityQueue retrieves elements according to priority.

The Queue interface provides two groups of methods:

Operation	Throws an exception on failure	Returns a special value on failure
Insert	add(e)	offer(e)
Retrieve without removing	element()	peek()
Retrieve and remove	remove()	poll()

```
Queue<String> queue = new ArrayDeque<>();
queue.offer("first");
queue.offer("second");
System.out.println(queue.poll()); // first
```

25. What is the difference between add() and offer() in a Queue?

Feature	add()	offer()
Purpose	Inserts an element into the queue	Inserts an element into the queue
If insertion succeeds	Returns true	Returns true
If insertion fails	Throws an exception	Returns false

Feature	add()	offer()
Best suited for	When failure should be reported as an exception	When you want to handle insertion failure with a return value

The difference matters mainly for capacity-restricted queues. With an unbounded queue such as ArrayDeque, insertion normally succeeds.

```
Queue<String> queue = new ArrayDeque<>();

queue.add("task A");

boolean added = queue.offer("task B");
```

26. What is the difference between poll() and remove() in a Queue?

Feature	poll()	remove()
Purpose	Retrieves and removes the queue's head	Retrieves and removes the queue's head
When queue has elements	Returns and removes the head element	Returns and removes the head element
When queue is empty	Returns null	Throws NoSuchElementException
Queue state after success	Head element is removed	Head element is removed
Empty-queue handling	Signals emptiness with a return value	Signals emptiness with an exception
Typical use	When an empty queue is a normal possibility	When an empty queue should be treated as an error
Return type	E (the queue's element type)	E (the queue's element type)
Example	String item = queue.poll();	String item = queue.remove();

27. What is the difference between peek() and element()?

Feature	peek()	element()
Purpose	Retrieves the queue's head without removing it	Retrieves the queue's head without removing it
If the queue is not empty	Returns the head element	Returns the head element
If the queue is empty	Returns null	Throws NoSuchElementException

Feature	peek()	element()
Does it remove the head?	No	No
Typical use	When an empty queue is expected and you want to check for null	When an empty queue should be treated as an error
Queue state after the call	Unchanged	Unchanged
Example	String next = queue.peek();	String next = queue.element();

28. What is a Map in Java?

A Map stores key-value associations. Each key is unique within the map, while values may repeat.

Map<String, Integer> ages = new HashMap<>();

ages.put("Asha", 25);

ages.put("Ravi", 25); // duplicate value is allowed

A map lets you find a value using its key:

int age = ages.get("Asha");

If you put a new value for an existing key, the new value replaces the old one. Map is part of the Collection Framework but does not extend the Collection interface.

29. What is the difference between a Map and a Set?

Feature	Map	Set
Stores	Key–value pairs	Individual elements
Example	"Asha" → 90	"Asha", "Ravi"
Uniqueness rule	Keys must be unique	Elements must be unique
Duplicate values/elements	Values may repeat	Duplicate elements are not retained
How you look up data	Use a key to retrieve its value	Check whether an element exists
Main operations	put(), get(), remove(), containsKey()	add(), contains(), remove()
Access by position	No index-based access	No index-based access
Ordering	Depends on implementation: HashMap has no guaranteed order; TreeMap sorts by key	Depends on implementation: HashSet has no guaranteed order; LinkedHashSet

Feature	Map	Set
		preserves insertion order; TreeSet sorts elements
Common implementations	HashMap, LinkedHashMap, TreeMap	HashSet, LinkedHashSet, TreeSet
Example declaration	Map<String, Integer> scores = new HashMap<>();	Set<String> names = new HashSet<>();
Relationship	Part of the Collection Framework, but does not extend Collection	Extends the Collection interface

30. Does a HashMap allow duplicate keys?

No. A HashMap cannot have two separate entries with equal keys. If you put a value for a key that already exists, the previous value is replaced.

```
Map<String, Integer> scores = new HashMap<>();

scores.put("Asha", 80);

scores.put("Asha", 95);

System.out.println(scores.get("Asha")); // 95
```

For custom key classes, equals() and hashCode() determine whether keys are treated as the same key. Those methods need to be implemented consistently.

31. Can a HashMap contain duplicate values?

Yes. Values do not need to be unique.

```
Map<String, Integer> scores = new HashMap<>();

scores.put("Asha", 90);

scores.put("Ravi", 90);
```

Both keys can map to the same value. A map restricts duplicate keys, not duplicate values.

32. What is the difference between HashMap and LinkedHashMap?

Feature	HashMap	LinkedHashMap
Key-value storage	Stores key-value pairs	Stores key-value pairs
Duplicate keys	Not allowed; a new value replaces the old value for that key	Same
Iteration order	No order is guaranteed	Preserves insertion order by default
Access order	Not maintained	Can be configured to maintain access order

Feature	HashMap	LinkedHashMap
Performance	Usually slightly faster	Usually slightly slower because it tracks order
Memory usage	Usually lower	Slightly higher because it maintains links between entries
null keys and values	Allows one null key and multiple null values	Also allows one null key and multiple null values
Common use	When ordering does not matter	When predictable iteration order is needed
Example	Map<String, Integer> map = new HashMap<>();	Map<String, Integer> map = new LinkedHashMap<>();

33. What is the difference between HashMap and TreeMap?

Feature	HashMap	TreeMap
Key order	No guaranteed order	Keeps keys sorted
How keys are ordered	Not ordered	Natural order or a supplied Comparator
Key requirements	Keys should implement hashCode() and equals() consistently	Keys must be comparable, or the map needs a Comparator
null keys	Allows one null key	Usually does not allow null keys with natural ordering
null values	Allowed	Allowed
Range operations	No built-in sorted range operations	Supports operations such as firstKey(), lastKey(), and subMap()
Common use	Fast lookup when key order does not matter	Sorted keys or key-range queries
Example	Map<String, Integer> map = new HashMap<>();	Map<String, Integer> map = new TreeMap<>();

34. What is the purpose of keySet(), values(), and entrySet()?

These methods return views of the contents of a map:

- keySet() provides the map's keys as a Set.
- values() provides the map's values as a Collection.
- entrySet() provides the map's key-value pairs as Map.Entry objects.

Map<String, Integer> scores = new HashMap<>();

```
scores.put("Asha", 90);
scores.put("Ravi", 85);
for (String key : scores.keySet()) {
  System.out.println(key);
}
for (Integer value : scores.values()) {
  System.out.println(value);
}

for (Map.Entry<String, Integer> entry : scores.entrySet()) {
  System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

35. What is an Iterator, and why is it used with collections?

An Iterator provides a standard way to visit elements in a collection without depending on its internal storage. Its main methods are:

- hasNext() — checks whether another element is available.
- next() — returns the next element.
- remove() — optionally removes the last element returned, if the collection supports it.

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi"));
Iterator<String> iterator = names.iterator();
while (iterator.hasNext()) {
  String name = iterator.next();
  System.out.println(name);
}
```

An iterator is particularly useful when removing elements during traversal. Calling list.remove() directly inside an ordinary enhanced for loop can cause a ConcurrentModificationException. Use the iterator's own remove() method when that removal pattern is supported.

B. Strings in Java (15 Questions)

36. What is a String in Java?

A String is an object that represents a sequence of characters. Java strings are instances of the String class.

```
String language = "Java";
String greeting = new String("Hello");
```

Strings are objects, not primitive values. String literals such as "Java" are commonly stored in a shared string pool, and a String cannot be modified after it is created.

37. Why is String immutable in Java?

An immutable object cannot be changed after it is created. Any operation that appears to modify a string returns a new string instead.

```
String text = "hello";
text.toUpperCase();
System.out.println(text); // hello
text = text.toUpperCase();
System.out.println(text); // HELLO
```

Immutability is useful because it makes strings safer to share, allows string literals to be reused, and helps strings behave predictably when used as keys in collections such as HashMap. If a key could change after insertion, its hash-based lookup behavior could become unreliable.

38. What is the difference between String and StringBuilder?

Feature	**String**	**StringBuilder**
Mutability	Immutable; its contents cannot be changed	Mutable; its contents can be changed
When text is modified	Operations such as concatenation create a new String	Methods such as append() modify the builder
Repeated updates	Can create many temporary objects	Usually more efficient for repeated changes
Thread safety	Safe to share because it is immutable	Not synchronized; not thread-safe for concurrent modification
Common operations	concat(), substring(), replace()	append(), insert(), delete(), reverse()
Converting to text	It is already a String	Call toString() to get a String
Best suited for	Text that is created once or rarely changed	Building or modifying text repeatedly
Example	String text = "Hello" + " Java";	StringBuilder text = new StringBuilder("Hello"); text.append(" Java");

39. What is the difference between StringBuilder and StringBuffer?

Feature	**StringBuilder**	**StringBuffer**
Mutability	Mutable; contents can be changed	Mutable; contents can be changed
Synchronization	Methods are not synchronized	Methods are synchronized

Feature	StringBuilder	StringBuffer
Thread safety	Not thread-safe for simultaneous access by multiple threads	Individual synchronized methods are thread-safe
Performance	Usually faster when used by one thread	May be slower because of synchronization overhead
Common methods	append(), insert(), delete(), reverse()	Provides similar methods
Best suited for	Building text in a single-threaded context	Shared use across threads when synchronized methods are appropriate
Example	StringBuilder sb = new StringBuilder("Hi");	StringBuffer sb = new StringBuffer("Hi");

40. What is the difference between == and equals() when comparing Strings?

Feature	==	equals()
Compares	Whether two references point to the same object	Whether two strings contain the same character sequence
Checks object identity?	Yes	No
Checks string content?	No	Yes
Result for separate objects with identical text	Usually false	true
Result for the same object reference	true	true
Effect of the string pool	Pooled literals may share a reference, so == can be true	Compares content regardless of pooling
Recommended for comparing string text?	No	Yes
Example	a == b	a.equals(b)

41. What is the String pool?

The string pool is a special area managed by the JVM where string literals and certain other strings can be reused.

```
String first = "Java";
String second = "Java";
System.out.println(first == second); // typically true: same pooled literal
```

A string created explicitly with new String(...) is a separate object:

String third = new String("Java");

System.out.println(first == third); // false

Use equals() to compare text regardless of whether strings are pooled. The intern() method can return the pooled representation of a string, but it is rarely necessary in ordinary application code.

42. What is the use of the length() method?

length() returns the number of UTF-16 code units in a String.

String word = "Java";

System.out.println(word.length()); // 4

For basic English text, this usually matches the number of visible characters. Some Unicode characters use more than one UTF-16 code unit, so length() may not equal the number of user-perceived characters.

For example, to count Unicode code points rather than UTF-16 code units, use:

int count = word.codePointCount(0, word.length());

43. What is the use of the charAt() method?

charAt(index) returns the UTF-16 char at the specified index. Indexes begin at 0.

String word = "Java";

char first = word.charAt(0); // 'J'

The index must be from 0 through length() - 1; otherwise, charAt() throws IndexOutOfBoundsException. Since some Unicode characters need two char values, use code-point methods when processing arbitrary Unicode text.

44. What is the use of the substring() method?

substring() extracts part of a string and returns it as a new String.

String text = "Collection";

String part = text.substring(3); // "lection"

String section = text.substring(0, 4); // "Coll"

In substring(beginIndex, endIndex), the start is included and the end is excluded. Both indexes must be valid; otherwise, the method throws IndexOutOfBoundsException.

45. What is the difference between equals() and equalsIgnoreCase()?

Feature	**equals()**	**equalsIgnoreCase()**
Compares	String contents, including letter case	String contents while ignoring letter case
Case sensitivity	Case-sensitive	Case-insensitive
"Java" vs. "java"	Returns false	Returns true

Feature	equals()	equalsIgnoreCase()
Other characters	Must match exactly	Must match; only letter case is ignored
Return type	boolean	boolean
Example	"Hello".equals("hello") → false	"Hello".equalsIgnoreCase("hello") → true
Typical use	Exact text comparison	Simple case-insensitive text comparison

46. What is the use of contains()?

contains(sequence) checks whether a string contains the specified character sequence. It returns a boolean.

```
String message = "Learning Java collections";

boolean found = message.contains("Java"); // true
```

The check is case-sensitive: "Java" and "java" are different sequences. contains() does not use regular expressions.

47. What is the use of indexOf() and lastIndexOf()?

indexOf() returns the index of the first occurrence of a character or substring. lastIndexOf() returns the index of the last occurrence. Both return -1 if there is no match.

```
String text = "banana";

System.out.println(text.indexOf("an"));     // 1

System.out.println(text.lastIndexOf("an")); // 3

System.out.println(text.indexOf("z"));      // -1
```

These methods are useful for locating separators, repeated text, or particular characters within a string.

48. What is the difference between replace() and replaceAll()?

Feature	replace()	replaceAll()
Search pattern	Literal character or character sequence	Regular expression (regex)
Special regex characters	Treated literally	Have regex meaning unless escaped
Replaces	Matching characters or text	Every substring matching the regex
Common use	Simple text replacement	Pattern-based replacement
Example	"a.b".replace(".", "-") → "a-b"	"a.b".replaceAll("\\.", "-") → "a-b"

Feature	replace()	replaceAll()
Invalid regex	Not applicable	Throws PatternSyntaxException
Performance	Usually simpler for literal replacement	Regex processing may add overhead

49. What is the use of trim()?

trim() returns a string with leading and trailing characters at or below the traditional ASCII space character removed.

```
String input = "   Java   ";
String cleaned = input.trim(); // "Java"
```

It does not change the original string. For broader Unicode whitespace handling, strip() and related methods are available in modern Java versions.

50. How would you reverse a String in Java?

For a straightforward reversal, use StringBuilder.reverse():

```
String original = "Java";
String reversed = new StringBuilder(original).reverse().toString();
System.out.println(reversed); // avaJ
```

A loop is another option:

```
String original = "Java";
StringBuilder reversed = new StringBuilder();
  for (int i = original.length() - 1; i >= 0; i--) {
   reversed.append(original.charAt(i));
}
    System.out.println(reversed); // avaJ
```